Laboratorium 1
Symulacje deterministyczne

Imię i nazwisko: Franciszek Jarek, Marcin Napieralski

1. Wstęp

Zaimplementowaliśmy algorytm Runge-Kutty 4(5) rzędu z adaptacyjnym krokiem całkowania. Użyliśmy wbudowanej funkcji solve_ivp z biblioteki SciPy, czyli metoda oparta o algorytm Dormanda-Prince'a.

Dokładność obliczeń kontrolowana jest przez parametry:

- atol = 1e-6 (tolerancja bezwzględna)
- rtol = 1e-6 (tolerancja względna)

Wyniki przedstawiają symulację dynamiki białek **p53, MDMcyto, MDMn oraz PTEN** w czterech scenariuszach opisujących różne stany komórki: **stan podstawowy, uszkodzenie DNA, stan nowotworowy oraz działanie terapii siRNA.**

Użyte biblioteki: numpy, matplotlib.pyplot, scipy, os.

Parametry:

- p53: 20 nM
- MDMcyto: 100 nM
- MDMn: 200 nM
- PTEN: 300 nM
- Całkowity czas symulacji: 48h

2. Implementacja modelu

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import solve_ivp
import os


# Parametry modelu
p1 = 8.8
p2_base = 440
p3_base = 100
d1 = 1.375e-14
d2_base = 1.375e-4
d3 = 3e-5
k1 = 1.925e-4
k2 = 1e5
k3 = 1.5e5

# Czynniki scenariuszy
value_siRNA = 0.02
value_PTEN_off = 0.0
```

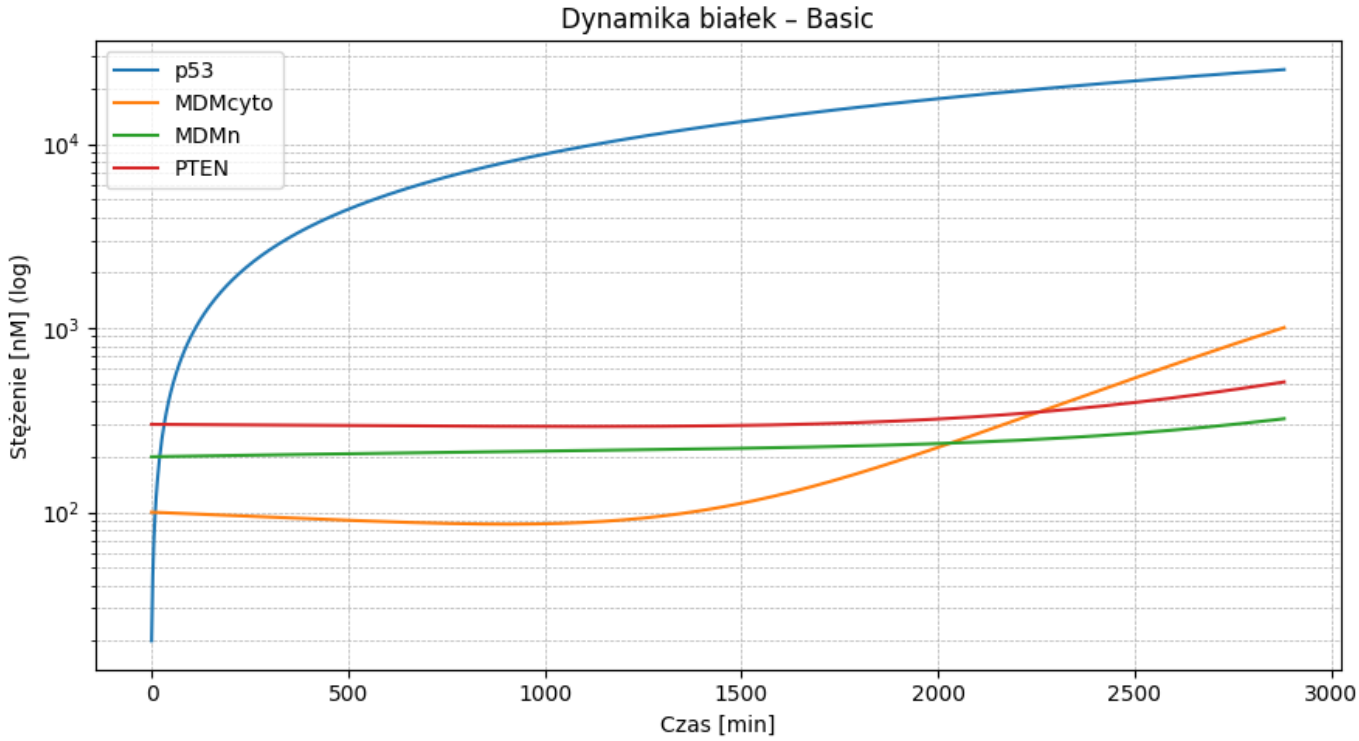
```python
value_no_DNA_damage = 0.1

# Definicja modelu
def model(t, y, siRNA=False, pten_off=False, no_DNA_damage=False):
    p53, mdmcyto, mdmn, pten = y
    dp53 = p1 - d1 * p53 * (mdmn**2)

    siRNA_factor = value_siRNA if siRNA else 1.0
    p2 = p2_base * siRNA_factor
    DNA_factor = value_no_DNA_damage if no_DNA_damage else 1.0
    d2 = d2_base * DNA_factor
    dMDMcyto = (p2 * p53**4 / (p53**4 + k2**4)
                - k1 * (k3**2)/(k3**2 + pten**2) * mdmcyto
                - d2 * mdmcyto)

    dMDMn = (k1 * (k3**2)/(k3**2 + pten**2) * mdmcyto - d2 * mdmn)

    p3 = 0.0 if pten_off else p3_base
    dPTEN = p3 * (p53**4)/(p53**4 + k2**4) - d3 * pten

    return [dp53, dMDMcyto, dMDMn, dPTEN]
```

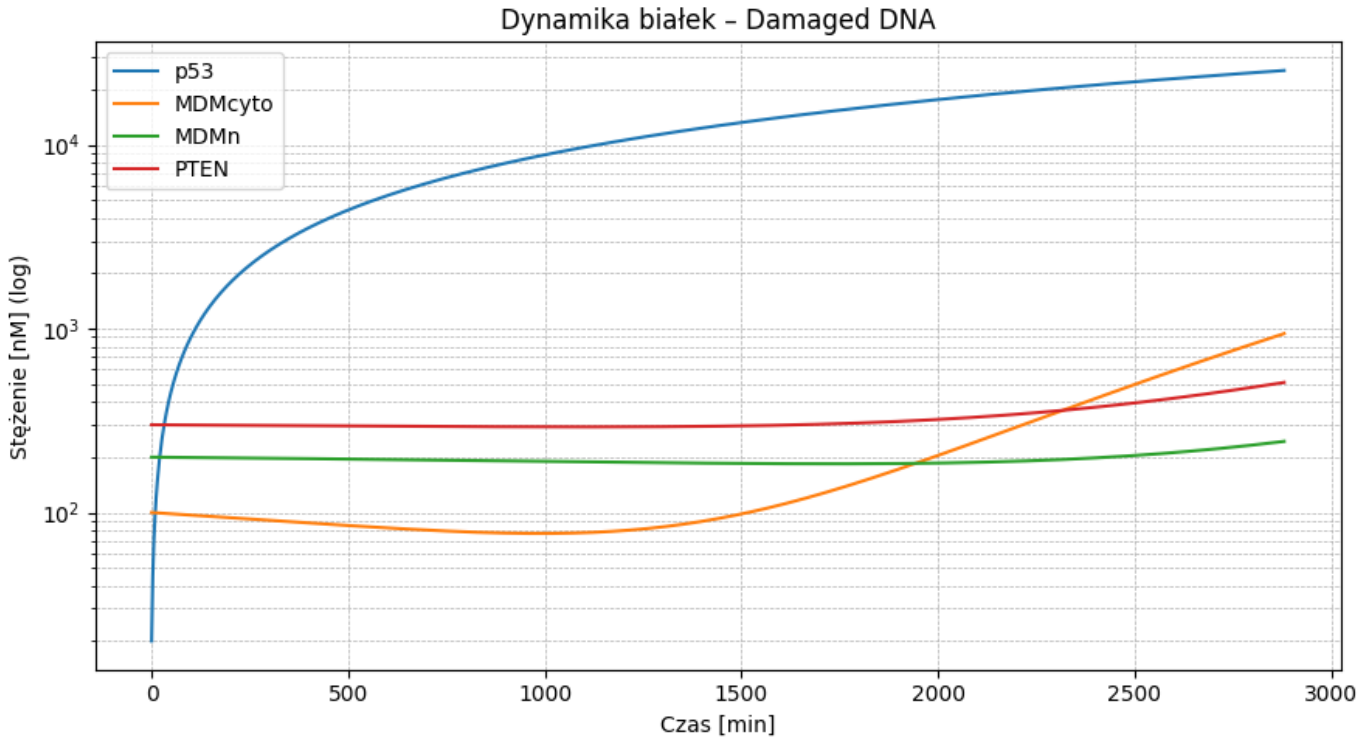
3. Symulacja

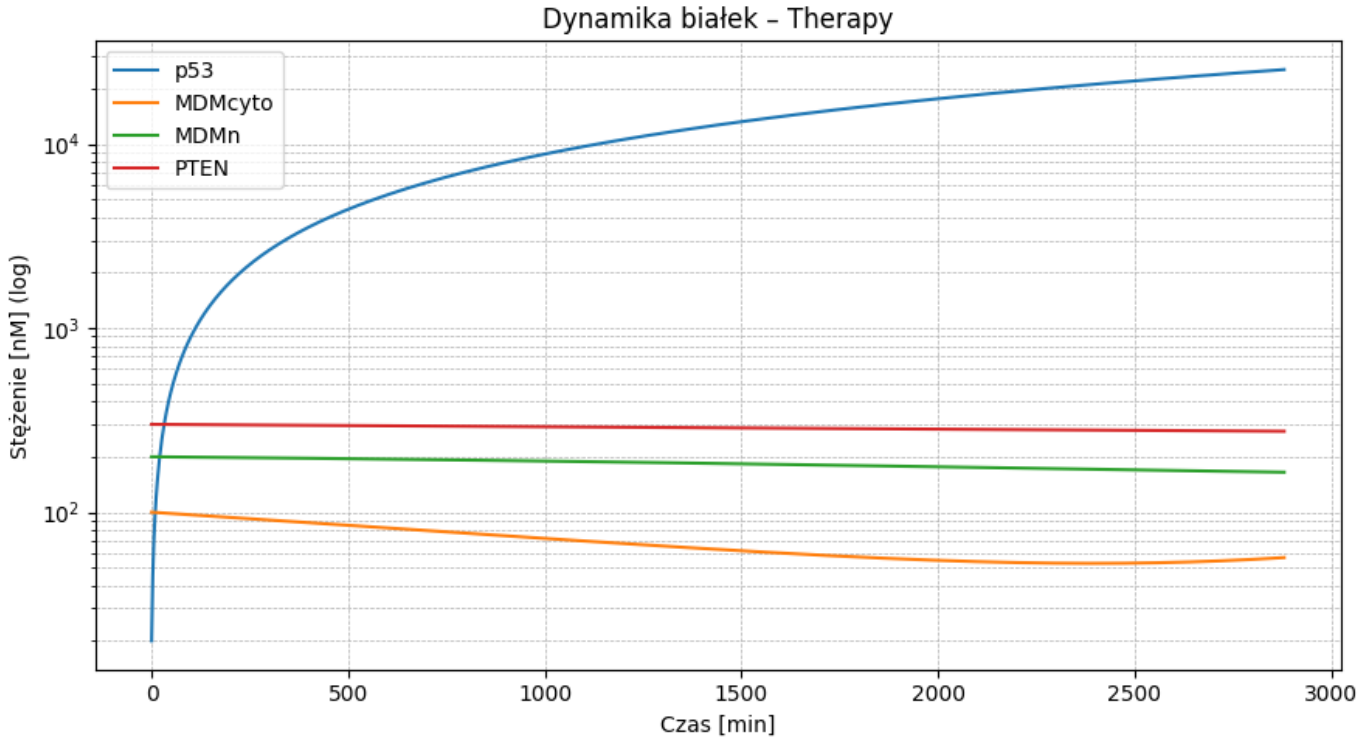
```python
# Symulacja
def simulate(end_time_min=48*60, atol=1e-6, rtol=1e-6):
    init = [20.0, 100.0, 200.0, 300.0]
    t_eval = np.linspace(0, end_time_min, 1000)
    conditions = {
        "Basic":       (False, False, True),
        "Damaged DNA": (False, False, False),
        "Tumor":       (False, True, False),
        "Therapy":     (True,  True, False),
    }
    results = {}

    for name, (siRNA, pten_off, no_DNA) in conditions.items():
        sol = solve_ivp(
            fun=lambda t, y: model(t, y, siRNA, pten_off, no_DNA),
            t_span=(0, end_time_min),
            y0=init,
            method='RK45',
            t_eval=t_eval,
            atol=atol,
            rtol=rtol
        )
        results[name] = (sol.t, sol.y)
    return results

# Wykresy
if __name__ == "__main__":
    sim = simulate()
```

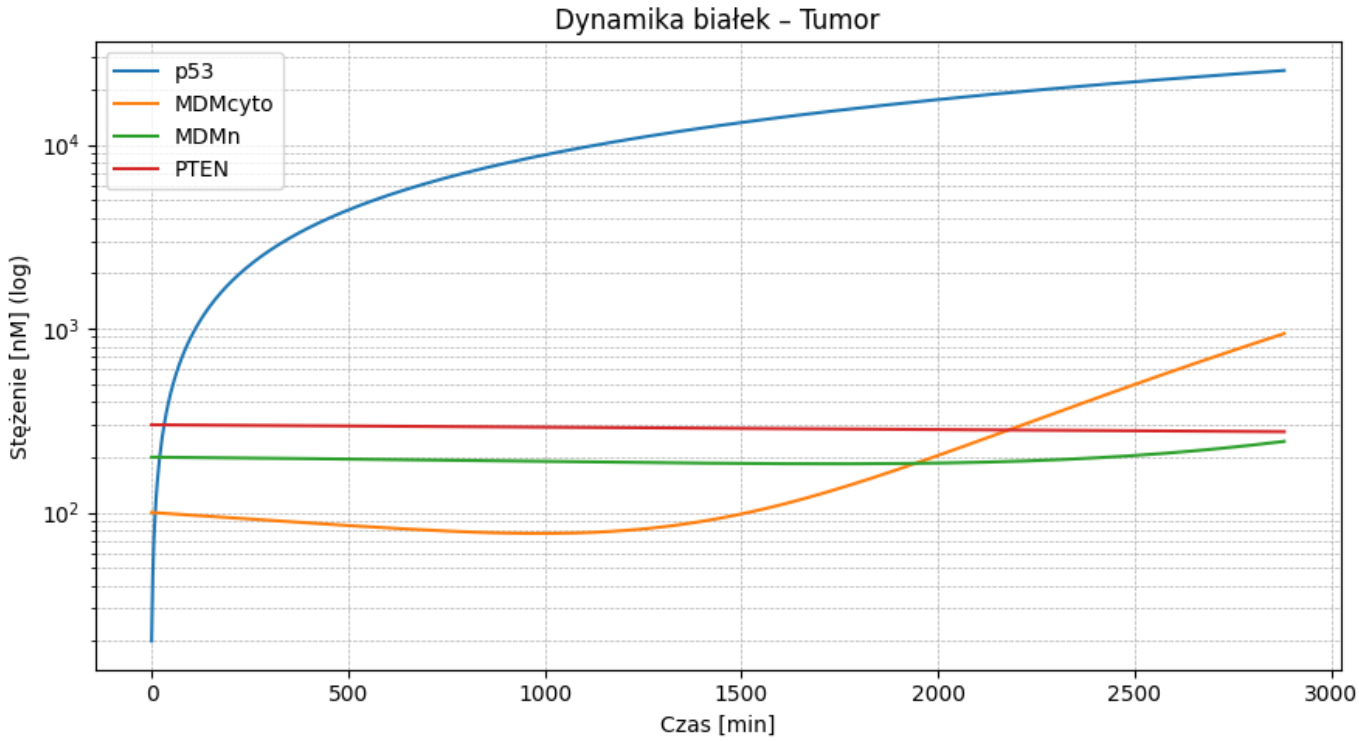
```
    labels = ["p53", "MDMcyto", "MDMn", "PTEN"]

    output_folder = "results"
    os.makedirs(output_folder, exist_ok=True)

    for name, (times, traj) in sim.items():
        plt.figure(figsize=(9, 5))
        for i, lbl in enumerate(labels):
            plt.plot(times, traj[i], label=lbl)
        plt.xlabel("Czas [min]")
        plt.ylabel("Stężenie [nM] (log)")
        plt.yscale("log")
        plt.title(f"Dynamika białek – {name}")
        plt.legend()
        plt.grid(True, which="both", linestyle="--", linewidth=0.5)
        plt.tight_layout()

        filename = f"{name.replace(' ', '_').lower()}.png"
        plt.savefig(os.path.join(output_folder, filename))
        plt.close()
```

4. Wyniki




Dynamika białek – Damaged DNA
p53
MDMcyto
MDMn
PTEN
Stężenie [nM] (log)
10^4
10^3
10^2
0
500
1000
1500
2000
2500
3000
Czas [min]


Dynamika białek – Therapy
p53
MDMcyto
MDMn
PTEN
Stężenie [nM] (log)
10^4
10^3
10^2
0
500
1000
1500
2000
2500
3000
Czas [min]



5. Wnioski

- Zastosowanie RK4 z adaptacyjnym krokiem pozwala dokładnie odwzorować dynamikę zmian zmiennych biologicznych przy zachowaniu efektywności obliczeniowej.
- Największe zmiany obserwowano w scenariuszu terapii (siRNA załączone, PTEN wyłączone, uszkodzenia DNA załączone) – duży spadek MDMcyto.
- Pozostałe wykresy nie różnią sie znacząco między sobą – może to wynikać z przyjętych danych wejściowych.

Cały kod znajduje się w pliku main.py oraz załączyliśmy wykonywalny plik main.exe, którego uruchomienie generuje folder z wykresami. Załączyliśmy również plik requirements.txt z potrzebnymi bibliotekami do prawidłowego działania kodu.